

Results of QSM-Cutoff on 19 Protocols

Consensus

```
# Signature
type value
relation chosen(V:value)

# Cutoff formula Rc
invariant [invar_0] forall V0,V1. ~chosen(V1) | ~chosen(V0) | V0 = V1
```

naive_consensus

```
# Signature
type node
type quorum
type value
relation member(N:node, Q:quorum)
relation vote(N:node, V:value)
relation decide(Q:quorum, V:value)
relation decision(V:value)

# Cutoff formula Rc
invariant [invar_1] forall N0,V0,V1. ~vote(N0,V1) | ~vote(N0,V0) | V0 = V1
invariant [invar_11] forall V0. (exists Q0. decide(Q0,V0) | ~decision(V0))
invariant [invar_4] forall N0,V0,Q0. vote(N0,V0) | ~decide(Q0,V0) | ~member(N0,Q0)
```

```
# Signature
type node
type value
type quorum
relation member(N:node, Q:quorum)
relation vote(N:node, V:value)
relation decision(V:value)
relation didNotVote(N:node) = forall V. ~vote(N, V)
relation chosenAt(Q:quorum, V:value) = forall N. member(N, Q) -> vote(N, V)

# Cutoff formula Rc
invariant [def_didNotVote] forall N. didNotVote(N) = (forall V. ~vote(N,V))
invariant [def_chosenAt] forall V,Q. chosenAt(Q,V) = (forall N. member(N,Q) -> vote(N,V))
invariant [invar_0] forall V0,V1,N0. ~vote(N0,V0) | ~vote(N0,V1) | V0 = V1
invariant [invar_22] forall V0. (exists Q0. chosenAt(Q0,V0) | ~decision(V0))
```

toy_consensus_forall

```
# Signature
type node
type quorum
type value
relation member(N:node, Q:quorum)
relation voted(N:node)
relation vote(N:node, V:value)
relation decided(V:value)
function voting_quorum: quorum

# Cutoff formula Rc
invariant [invar_2] forall V0,N0. voted(N0) | ~vote(N0,V0)
invariant [invar_3] forall V0,N0,V1. ~vote(N0,V0) | ~vote(N0,V1) | V0 = V1
invariant [invar_22] forall N0. (exists V0. ~voted(N0) | vote(N0,V0))
invariant [invar_4] forall V0,N0,Q0. ~decided(V0) | ~member(N0,Q0)
                                     | voting_quorum ~= Q0 | vote(N0,V0)
```

```
# Signature
type node
type quorum
type value
relation member(N:node, Q:quorum)
relation voted(N:node)
relation vote(N:node, V:value)
relation decided(V:value)
relation chosenAt(Q:quorum, V:value) = forall N. member(N, Q) -> vote(N, V)

# Cutoff formula Rc
invariant [def_chosenAt] forall Q,V. chosenAt(Q,V) = (forall N. member(N,Q) -> vote(N,V))
invariant [invar_0] forall NO,V0,V1. ~vote(NO,V0) | ~vote(NO,V1) | V0 = V1
invariant [invar_22] forall NO. (exists V0. vote(NO,V0) | ~voted(NO))
invariant [invar_7] forall NO,V0. ~vote(NO,V0) | voted(NO)
invariant [invar_21] forall V0. (exists Q0. chosenAt(Q0,V0) | ~decided(V0))
```

```

# Signature
type node
type value
type quorum
relation member(N:node, Q:quorum)
relation vote_request_msg(N0:node, N1:node)
relation voted(N:node)
relation vote_msg(N0:node, N1:node)
relation votes(N0:node, N1:node)
relation leader(N:node)
relation decided(N:node, V:value)
relation chosen(N:node) = exists Q. forall Nv. member(Nv, Q) -> votes(N, Nv)

# Cutoff formula Rc
invariant [def_chosen] forall N. chosen(N) = (exists Q. (forall Nv. member(Nv,Q) -> votes(N,Nv)))
invariant [invar_8] forall N1,V1,V0,N0. ~decided(N0,V0) | ~decided(N1,V1) | (N0 = N1 & V0 = V1)
invariant [invar_105] forall N1. ~voted(N1) | vote_msg(N1,N1) | (exists N0. (vote_msg(N1,N0) & N0 ~= N1))
invariant [invar_11] forall V0,N0. leader(N0) | ~decided(N0,V0)
invariant [invar_12] forall N1,N0. ~vote_msg(N0,N1) | voted(N0)
invariant [invar_13] forall N2,N1,N0. ~vote_msg(N0,N1) | ~vote_msg(N0,N2) | ((N1 = N2 & N0 ~= N1) | (N0 = N2 & N0 = N1))
invariant [invar_1] forall N1,N0. ~votes(N0,N1) | vote_msg(N1,N0)
invariant [invar_4] forall N0. ~leader(N0) | chosen(N0)
invariant [invar_69] forall N1,N0. ~vote_msg(N0,N1) | vote_request_msg(N1,N0) | vote_request_msg(N0,N1) | N0 = N1
invariant [invar_56] forall N1,N0. ~vote_msg(N0,N1) | vote_request_msg(N1,N0) | vote_msg(N1,N0) | N0 = N1

```

lock_server

```
# Signature
type client
type server
relation link(X: client, Y: server)
relation semaphore(X: server)

# Cutoff formula Rc
invariant [invar_0] forall S0,C0. ~semaphore(S0) | ~link(C0,S0)
invariant [invar_1] forall S0,C0,C1. ~link(C1,S0) | ~link(C0,S0) | C0 = C1
invariant [invar_2] forall S0. (exists C0. semaphore(S0) | link(C0,S0))
```



```
# Signature
type node
relation lock_msg(N:node)
relation grant_msg(N:node)
relation unlock_msg(N:node)
relation holds_lock(N:node)
individual server_holds_lock: bool

# Cutoff formula Rc
invariant [invar_0] forall N0. ~server_holds_lock | ~grant_msg(N0)
invariant [invar_1] forall N0. ~server_holds_lock | ~unlock_msg(N0)
invariant [invar_2] forall N0. ~server_holds_lock | ~holds_lock(N0)
invariant [invar_3] forall N1,N0. ~grant_msg(N0) | ~holds_lock(N1)
invariant [invar_4] forall N1,N0. ~unlock_msg(N1) | ~holds_lock(N0)
invariant [invar_5] forall N1,N0. ~holds_lock(N1) | ~holds_lock(N0) | N0 = N1
invariant [invar_6] forall N1,N0. ~grant_msg(N1) | ~grant_msg(N0) | N0 = N1
invariant [invar_7] forall N1,N0. ~unlock_msg(N1) | ~grant_msg(N0)
invariant [invar_8] forall N1,N0. ~unlock_msg(N1) | ~unlock_msg(N0) | N0 = N1
invariant [invar_9] exists N0. holds_lock(N0) | grant_msg(N0) | server_holds_lock | unlock_msg(N0)
```

lockserv_automaton

```
# Signature
type node
relation lock_msg(N:node)
relation unlock_msg(N:node)
relation grant_msg(N:node)
relation holds_lock(N: node)
relation held

# Cutoff formula Rc
invariant [invar_0] forall N1,N0. ~holds_lock(N0) | ~unlock_msg(N1)
invariant [invar_1] forall N0. ~holds_lock(N0) | held
invariant [invar_2] forall N1,N0. ~grant_msg(N0) | ~holds_lock(N1)
invariant [invar_3] forall N1,N0. ~holds_lock(N0) | ~holds_lock(N1) | N0 = N1
invariant [invar_4] forall N0. held | ~unlock_msg(N0)
invariant [invar_5] forall N0. held | ~grant_msg(N0)
invariant [invar_6] forall N1,N0. ~unlock_msg(N1) | ~grant_msg(N0)
invariant [invar_7] forall N1,N0. ~grant_msg(N1) | ~grant_msg(N0) | N0 = N1
invariant [invar_8] forall N1,N0. ~unlock_msg(N1) | ~unlock_msg(N0) | N0 = N1
invariant [invar_9] exists N0. ~held | grant_msg(N0) | unlock_msg(N0) | holds_lock(N0)
```

simple-decentralized-lock

```
# Signature
type node
relation message(Src: node, Dst: node)
relation has_lock(N: node)
individual start_node: node

# Cutoff formula Rc
invariant [invar_0] forall N2,N1,N0. ~has_lock(N0) | ~message(N1,N2)
invariant [invar_1] forall N2,N1,N3,N0. ~message(N1,N3) | ~message(N0,N2) | (N0 = N1 & N2 = N3)
invariant [invar_3] forall N1,N0. ~has_lock(N0) | ~has_lock(N1) | N0 = N1
invariant [invar_5] exists N1,N0. message(N0,N1) | has_lock(N0)
```

```
# Signature
type node
type request
type response
relation match(R:request, P:response)
relation request_sent(N:node, R:request)
relation response_sent(N:node, P:response)
relation response_received(N:node, P:response)
relation responseMatched(N:node, P:response) = exists R. request_sent(N,R) & match(R,P)

# Cutoff formula Rc
invariant [def_responseMatched] forall N,P. responseMatched(N,P)
                                     = (exists R. (request_sent(N,R) & match(R,P)))
invariant [invar_0] forall NO,R0. response_sent(NO,R0) | ~response_received(NO,R0)
invariant [invar_8] forall NO,R0. responseMatched(NO,R0) | ~response_sent(NO,R0)
```

```
# Signature
type node
relation requested(N1: node, N2:node)
relation replied(N1:node, N2:node)
relation holds(N:node)

# Cutoff formula Rc
invariant [eq_0] forall N. replied(N,N) = requested(N,N)
invariant [invar_0] forall N0. ~replied(N0,N0)
invariant [invar_1] forall N1,N0. replied(N0,N1) | ~holds(N0) | N0 = N1
invariant [invar_4] forall N1,N0. ~replied(N1,N0) | ~replied(N0,N1) | N0 = N1
```

```
# Signature
type key
type value
type node
relation table(N:node, K:key, V:value)
relation owner(N:node, K:key)
relation transfer_msg(N:node, K:key, V:value)

# Cutoff formula Rc
invariant [invar_0] forall K0,N1,N0,V0. ~owner(N0,K0) | ~transfer_msg(N1,K0,V0)
invariant [invar_2] forall K0,V0,N0,V1,N1. ~transfer_msg(N1,K0,V1) | ~transfer_msg(N0,K0,V0)
                                     | (V0 = V1 & N0 = N1)
invariant [invar_3] forall K0,N1,N0. ~owner(N0,K0) | ~owner(N1,K0) | N0 = N1
invariant [invar_4] forall K0,N0,V0. owner(N0,K0) | ~table(N0,K0,V0)
invariant [invar_6] forall K0,V0,N0,V1,N1. ~table(N1,K0,V1) | ~table(N0,K0,V0)
                                     | (V0 = V1 & N0 = N1)
```

sharded_kv_no_lost_keys

```
# Signature
type key
type value
type node
relation table(N:node, K:key, V:value)
relation owner(N:node, K:key)
relation transfer_msg(N:node, K:key, V:value)

# Cutoff formula Rc
invariant [invar_0] forall NO,V0,K0,N1. ~transfer_msg(N1,K0,V0) | ~owner(NO,K0)
invariant [invar_2] forall NO,V1,V0,N1,K0. ~transfer_msg(NO,K0,V0) | ~transfer_msg(N1,K0,V1)
                                     | (V0 = V1 & NO = N1)
invariant [invar_3] forall NO,K0,N1. ~owner(N1,K0) | ~owner(NO,K0) | NO = N1
invariant [invar_6] forall NO,V1,V0,N1,K0. ~table(NO,K0,V0) | ~table(N1,K0,V1)
                                     | (V0 = V1 & NO = N1)
invariant [invar_7] forall K0. (exists NO,V0. transfer_msg(NO,K0,V0) | owner(NO,K0))
invariant [invar_4] forall NO,V0,K0. ~table(NO,K0,V0) | owner(NO,K0)
```

```
# Signature
type node
relation internal(N:node)
relation sent(N0:node, N1:node)
relation allowed_in(N:node)

# Cutoff formula Rc
invariant [invar_0] forall N0. ~sent(N0,N0)
invariant [invar_10] forall N1. ~allowed_in(N1) | (exists N0. (sent(N0,N1) & N0 ~= N1))
invariant [invar_1] forall N0. ~internal(N0) | ~allowed_in(N0)
invariant [invar_7] forall N1,N0. internal(N0) | internal(N1) | ~sent(N0,N1) | N0 = N1
invariant [invar_8] forall N1,N0. allowed_in(N0) | ~sent(N0,N1) | allowed_in(N1) | N0 = N1
```


TCommit

```
# Signature
type resource_manager
relation working(R:resource_manager)
relation prepared(R:resource_manager)
relation committed(R:resource_manager)
relation aborted(R:resource_manager)

# Cutoff formula Rc
invariant [invar_0] forall R1,R0. ~committed(R0) | ~working(R1)
invariant [invar_1] forall R0. ~working(R0) | ~prepared(R0)
invariant [invar_2] forall R0. ~aborted(R0) | ~working(R0)
invariant [invar_3] forall R0. ~aborted(R0) | ~prepared(R0)
invariant [invar_4] forall R1,R0. ~aborted(R0) | ~committed(R1)
invariant [invar_5] forall R0. ~committed(R0) | ~prepared(R0)
invariant [invar_7] forall R0. working(R0) | committed(R0) | aborted(R0) | prepared(R0)
```

TwoPhase

```
# Signature
type resource_manager
relation working(R:resource_manager)
relation prepared(R:resource_manager)
relation committed(R:resource_manager)
relation aborted(R:resource_manager)
relation tm_init
relation tm_committed
relation tm_aborted
relation tm_prepared(R:resource_manager)
relation msg_prepared(R:resource_manager)
relation msg_commit
relation msg_abort
# Cutoff formula Rc
invariant [eq_0] msg_abort = tm_aborted
invariant [eq_1] msg_commit = tm_committed
invariant [invar_1] forall R0. ~aborted(R0) | ~working(R0)
invariant [invar_6] forall R0. ~tm_prepared(R0) | msg_prepared(R0)
invariant [invar_8] ~tm_init | ~msg_commit
invariant [invar_10] ~msg_commit | ~msg_abort
invariant [invar_12] ~tm_init | ~msg_abort
invariant [invar_14] forall R0. ~prepared(R0) | ~aborted(R0)
invariant [invar_21] forall R0. ~committed(R0) | ~prepared(R0)
invariant [invar_23] msg_abort | msg_commit | tm_init
invariant [invar_2] forall R0. ~msg_prepared(R0) | ~working(R0)
invariant [invar_16] forall R0. ~committed(R0) | msg_commit
invariant [invar_7] forall R0. ~msg_commit | tm_prepared(R0)
invariant [invar_44] forall R0. working(R0) | prepared(R0) | committed(R0) | aborted(R0)
invariant [invar_30] forall R0. ~msg_prepared(R0) | msg_abort | ~aborted(R0)
invariant [invar_11] forall R0. ~prepared(R0) | msg_prepared(R0)
```

simple-election

```
# Signature
type acceptor
type quorum
type proposer
relation member(X: acceptor, Y: quorum)
relation start(P: proposer)
relation promise(A: acceptor, P: proposer)
relation leader(P: proposer)

# Cutoff formula Rc
invariant [def_didNotPromise] forall A. didNotPromise(A) = (forall P. ~promise(A,P))
invariant [def_chosenAt] forall Q,P. chosenAt(Q,P) = (forall A. member(A,Q) -> promise(A,P))
invariant [invar_0] forall A0,P1,P0. ~promise(A0,P1) | ~promise(A0,P0) | P0 = P1
invariant [invar_24] forall P0. (exists Q0. ~leader(P0) | chosenAt(Q0,P0))
invariant [invar_2] forall A0,P0. start(P0) | ~promise(A0,P0)
```

quorum-leader-election-wo-maj

```
# Signature
type node
type nset
relation isleader(N:node)
relation voted(N:node, M:node)
relation member(N: node, Q: nset)
individual quorum : nset

# Cutoff formula Rc
invariant [invar_0] forall N2,N0,N1. ~voted(N0,N2) | ~voted(N0,N1)
                                     | ((N1 = N2 & N0 ~= N1) | (N0 = N1 & N0 = N2))
invariant [invar_3] forall N0,N1,NSET0. voted(N1,N0) | ~isleader(N0) | quorum ~= NSET0
                                     | ((N0 ~= N1 | ~member(N0,NSET0))
                                     & (N0 = N1 | ~member(N1,NSET0) | member(N0,NSET0))
                                     & (N0 = N1 | ~member(N0,NSET0) | ~member(N1,NSET0)))
```